- *Divide and conquer:* It repeatedly divides the search interval in half.
- *Midpoint check:* It compares the target value to the middle element of the current interval.
- *Narrowing the search:* If the target is less than the middle, the search continues in the left half; if greater, it continues in the right half.
- *Efficient:* Binary search is highly efficient, with a time complexity of O(log n), making it much faster than linear search for large datasets.

We can see the model has generated a response of five points according to the text prompt provided in the input.

Multimodal support: Multimodal processing, which is the ability to process images, audio and video content, is one of the biggest achievements of GenAI models. Extracting text from media, summarising information from audio or video files, and the ability to generate output based on the input media are significant use cases that are made possible by GenAI models. And thanks to Spring AI, integration of such capabilities into regular applications is possible with just a few lines of code.

Consider the following multi-model use case of generating text output from media. We provide a picture of the flight departure schedule to the model and ask it to analyse the image and provide the list of flights meeting a criterion. The Java code using Spring AI is:

```
String userPrompt = "Analyze the content in provided
image, and provide list of flights that are not delayed or
cancelled";
ClassPathResource resource = new ClassPathResource("/flight_
schedule.png ");
UserMessage message = new UserMessage(userPrompt, List.of(new
Media(MimeTypeUtils.IMAGE_PNG,resource)));
ChatResponse response = chatModel.call(new Prompt(List.
of(message)));
```

The image supplied to the above code snippet is shown in Figure 1.

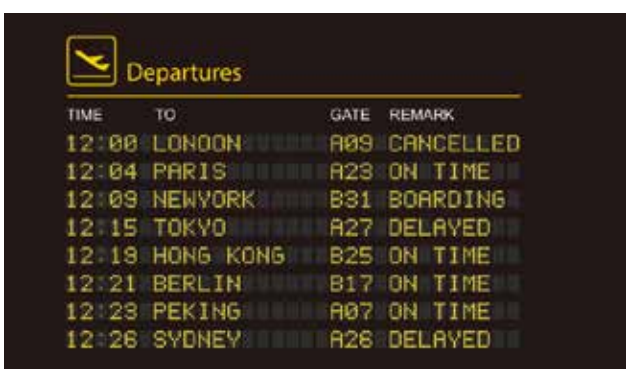

Figure 1: Image for content analysis

Sample output from the above code is:

```
The following flights are not delayed or cancelled:
- 12:04 PARIS (A23 ON TIME)
- 12:19 HONG KONG (B25 ON TIME)
- 12:21 BERLIN (B17 ON TIME)
- 12:23 PEKING (A07 ON TIME)
```

The power of the GenAI models is evident from the response generated above where the model parsed the media and extracted the information, and generated output text based on the instructions provided in the input prompt.

Support for RAG: Spring AI offers VectorStore abstraction, which can be used to interact with various vector databases. This makes it possible to add retrieval augmented generation (RAG) capabilities to Java applications. Advanced capabilities like RAG are very helpful for organisations in enriching the commoditised GenAI models with domain-specific and industry-specific knowledge. This makes the models more context-aware, and helps in generating tuned, custom results that are more useful compared to generic responses.

These are only a few of the key features provided by Spring AI. The community has also added a lot of critical supporting capabilities that makes the Spring AI framework truly enterprise-grade for use. Features like observability provide insights into AI related operations. Interfaces like 'Evaluator' help to evaluate AI models and can be used to protect applications from problems like hallucinations by AI models.

The Spring open source frameworks have served Java applications for decades. Their innovations, stability and support have immensely benefited the Java ecosystem. Spring AI is another great framework that is being enhanced iteratively with great features. This will help the Java developer ecosystem and organisations to quickly integrate their applications with the GenAI tech advancements. END

References

- *https://spring.io/projects/spring-ai*
- *https://docs.spring.io/spring-ai/reference/api/index.html*

By: Lokesh Chenta and Vijayabhaskar Reddy

Lokesh Chenta works as a principal II software architect at Sabre, India. He is experienced in the design and development of enterprise products and technology platforms that perform at high scale.

Vijayabhaskar Reddy works as a principal software architect at Sabre, India.